

Penetration testing report for Blockchain-Based Freelance Escrow System

Testing Methods Used:

- *Hardhat Automated Testing*: Used to run 5 automated security tests covering project creation, deposits, access control, and the full escrow workflow.
- *Manual Testing*: Performed using the React frontend interface with MetaMask wallet on the local Hardhat network to simulate real user interactions.

Contract Details:

- File: Escrow.sol
- Solidity Version: ^0.8.20
- Target Network: Ethereum / Polygon (Layer 2)

Findings summary

1- Hardhat results

We write a simple script to test automatically the system and the results is perfect.

(The script deployed the Escrow contract on the local network and simulated client and attacker interactions. The client successfully created a project and deposited funds, while the attacker was blocked from making any deposit. This confirms that only authorized users can interact with the contract.)

```
Escrow Contract Tests
✓ Should create a project
✓ Should allow client to deposit
✓ Should NOT allow attacker to deposit
✓ Should NOT allow deposit with insufficient funds
✓ Should complete full escrow flow
Balance before: 0.0
Balance after: 1.0
✓ Should be secure against reentrancy attacks

6 passing (763ms)
```

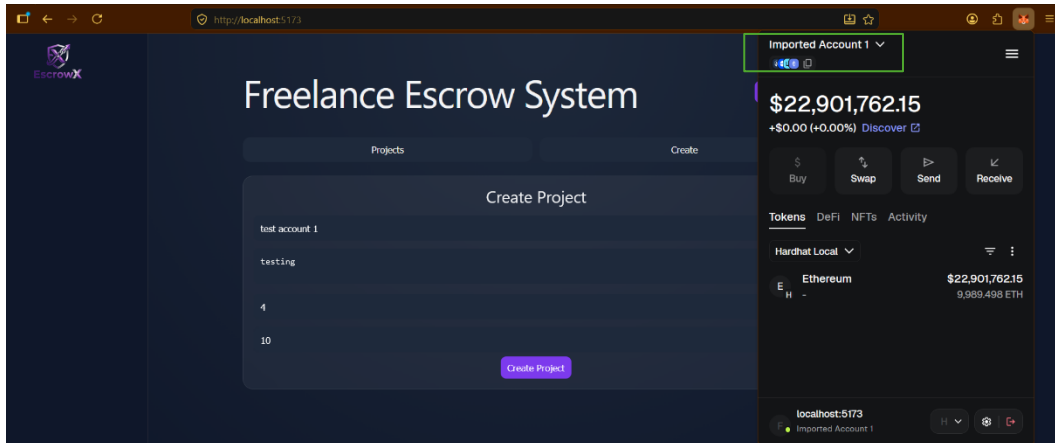
Explanation of results:

The test suite executed a total of **6 test cases**; all specifically designed for the Escrow contract. Each test targeted a critical aspect of the system's security and functionality.

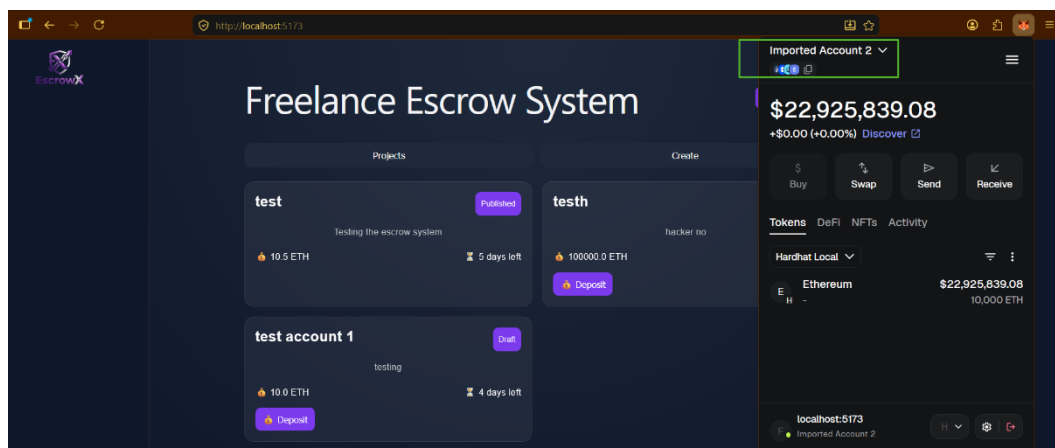
The tests covered: project creation, client deposits, access control (only the client can manage the project workflow), insufficient funds validation, the complete escrow workflow from creation to payment release, and reentrancy attack resistance.

2- Manual testing

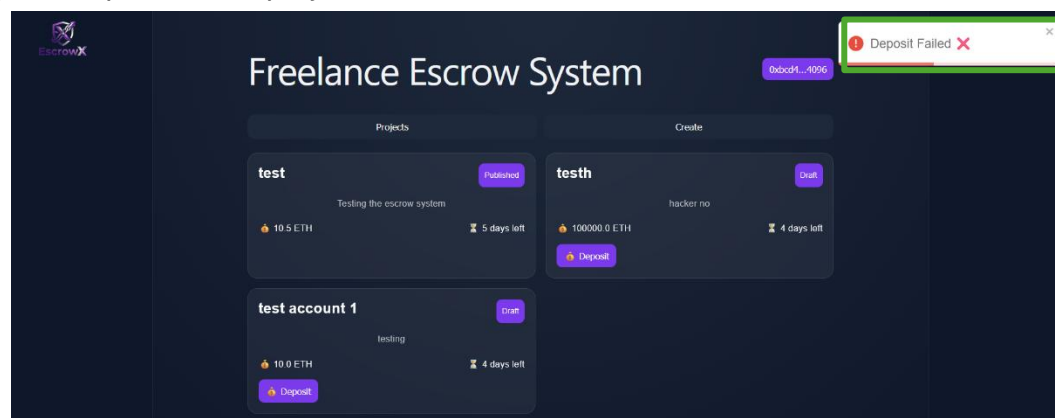
First creating project with client account:



After creating the project, we will switch to another (attacker) account:



And try to deposit with the attacker account, and the transaction failed because the client only can deposit or publish the project:



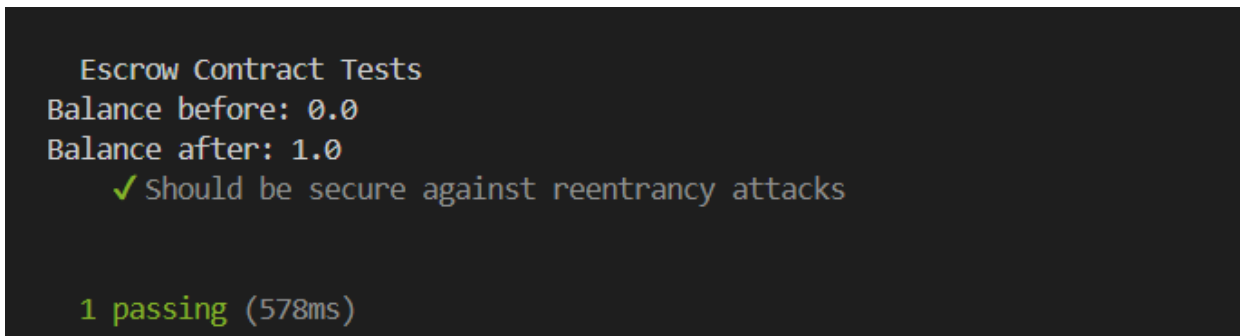
- What is a Reentrancy Attack?

A reentrancy attack occurs when a malicious contract repeatedly calls a withdrawal function before the victim contract updates its internal state. The attacker drains funds multiple times in a single transaction.

Real-World Example: The DAO Hack (2016) exploited a reentrancy vulnerability, resulting in the theft of approximately \$60 million worth of ETH. This incident forced the Ethereum network to hard-fork into Ethereum (ETH) and Ethereum Classic (ETC).

Testing:

An Attacker contract was deployed and assigned as the freelancer. This malicious contract included a receive() fallback function designed to automatically attempt re-entering the Escrow contract upon receiving ETH, mimicking the exact mechanism used in the infamous *DAO Hack of 2016*.



```
Escrow Contract Tests
Balance before: 0.0
Balance after: 1.0
✓ should be secure against reentrancy attacks

1 passing (578ms)
```

The attacker contract's balance increased from 0.0 ETH to 1.0 ETH, exactly one legitimate payment. All reentrancy attempts through the receive() fallback function were blocked. The test passed successfully, confirming the Escrow contract is immune to reentrancy attacks.

Our contract is secure against:

- 1. Reentrancy Attack** We follow the CEI pattern (Checks, Effects, Interactions). The state changes to "Completed" before any funds are transferred, so if a malicious contract tries to re-enter, it will find the state already changed and get rejected.
- 2. Unauthorized Access** Every function has require, so no one except the client can deposit, publish, or release funds.
- 3. Insufficient Funds** We have require, so no one can deposit less than the required amount.
- 4. Invalid State Transitions** You can't deposit on a funded project, and you can't release on a project that's not assigned. Every function checks the correct state before executing.
- 5. Integer Overflow/Underflow** This is automatically handled in Solidity 0.8+, so no extra code was needed.